Lab2: Partitioning

1. Introduction to Table Partitioning

Table partitioning is a database technique that divides a large table into smaller, more manageable pieces called **partitions**.
Although the table appears as a single object to users, SQL Server stores its data in separate partitions based on a specified column (typically a date or ID).

Partitioning is particularly useful for large transactional or time-series databases where data grows continuously, such as student records, financial transactions, or logs.

---

2. Core Components of Partitioning

Before creating a partitioned table, three essential elements must be defined:

- **Partition Function:** Determines how the data is divided (for example, by Department or date ranges).
- **Partition Scheme:** Maps each partition to a filegroup in the database.
- **Partitioned Table:** The table that stores its data based on the defined function and scheme.

3. Example Overview

In this lab, you will create a partitioned table named **Students** whose rows are partitioned by the **EnrollmentYear** column.

The steps are:

1. Define a partition function that determines the year boundaries.
2. Create a partition scheme mapping partitions to storage.
3. Create a partitioned table and insert sample data.
4. Verify partition placement and compare with a non-partitioned copy.
5. Observe execution plans.

4. Creating the Partition Function

A **partition function** defines how SQL Server divides rows among partitions.

```
CREATE PARTITION FUNCTION PF_By_Year (DATE)
AS RANGE LEFT FOR VALUES ('2019-12-31', '2020-12-31', '2021-12-31');
```

- The function divides data according to EnrollmentYear values.
- The RANGE LEFT keyword indicates that each boundary value belongs to the left partition.
- Three boundary values produce four partitions (n+1).

RANGE RIGHT and LEFT Comparison

Partition	Range Left	Range Right
1	<= 2019-12-31	< 2019-12-31
2	> 2019-12-31 and <= 2020-12-31	>= 2019-12-31 AND < 2020-12-31
3	> 2020-12-31 and <= 2021-12-	>= 2020-12-31 AND < 2021-12-31

Partition	Range Left	Range Right
	31	
4	> 2021-12-31	>= 2021-12-31

5. Creating the Partition Scheme

A partition scheme defines the physical storage locations of each partition.

```
CREATE PARTITION SCHEME PS_StudentYear
AS PARTITION PF_By_Year
ALL TO ([PRIMARY]); --Default File Group
```

In enterprise scenarios, each partition can be assigned to a separate filegroup to distribute data across multiple storage devices, improving performance and manageability.

6. Creating the Partitioned Table

The table must be created on the partition scheme, specifying the column that determines partition placement.

```
CREATE TABLE Students (
    StudentID INT,
    StudentName VARCHAR(100),
    EnrollmentYear DATE,
    CONSTRAINT PK_Students PRIMARY KEY CLUSTERED (EnrollmentYear, Stude
ntID)
)
ON PS_StudentYear(EnrollmentYear);
```

- The table is partitioned by the **EnrollmentYear** column.
- Each record is stored in the partition corresponding to its year.
- The **PRIMARY KEY** includes the partitioning column to ensure consistency and maintain clustering alignment. > **Note:** In SQL Server, the partitioning

column must be part of the clustered index key.
> This ensures that rows are physically organized according to the partition boundaries.

7. Inserting Sample Data

```
INSERT INTO dbo.Students (StudentID, StudentName, EnrollmentYear)
VALUES
 (1, 'Ahmed',   '2018-01-01'),
 (2, 'Mona',    '2019-05-10'),
 (3, 'Omar',    '2020-07-20'),
 (4, 'Sara',    '2021-11-12'),
 (5, 'Youssef', '2022-03-05');
```

8. Verifying Partition Distribution

The $PARTITION system function returns the partition number in which each record is stored.

```
SELECT
    $PARTITION.PF_By_Year(EnrollmentYear) AS PartitionNumber,*
FROM dbo.Students;
```

Expected Output:

PartitionNumber	StudentName	EnrollmentYear
1	Ahmed	2018-01-01
2	Mona	2019-05-10
3	Omar	2020-07-20
3	Sara	2021-11-12
4	Youssef	2022-03-05

Each record is stored in the partition corresponding to its `EnrollmentYear`, confirming that the partition function and scheme are working correctly.

9. Comparison with a Non-Partitioned Table

Objective: Show that **partitioning affects data access and performance**, not query results.

1. Create a Non-Partitioned Table for Reference

```
SELECT *
INTO dbo.Students_NonPartitioned
FROM dbo.Students;
-- Query the non-partitioned table
SELECT *
FROM dbo.Students_NonPartitioned
WHERE EnrollmentYear = '2020-01-01';

-- Query the partitioned table
SELECT *
FROM dbo.Students
WHERE EnrollmentYear = '2020-01-01';
```

- Both return identical results, confirming that **partitioning does not affect query semantics.**

3. Observing Query Performance

Partitioning does not change the logical results of a query, but it can significantly affect how SQL Server retrieves data.
When a table is partitioned by a column that is used in the query filter (for example, `WHERE EnrollmentYear = '2020-01-01'`), SQL Server can limit access to the relevant partition instead of scanning the entire table.

Enable **Include Actual Execution Plan** in SQL Server Management Studio (Ctrl + M) and execute both queries to observe the difference:

Table	Rows Read	Interpretation
Non-Partitioned	High – all rows	SQL Server scans all rows, increasing I/O and reducing efficiency.
Partitioned	Low –rows only	SQL Server reads only the target partition

*This confirms that **partitioning enhances query performance** for range-based filters while maintaining identical results.*

10. Filegroups in SQL Server

Definition

A **filegroup** in Microsoft SQL Server is a logical structure that organizes one or more physical data files within a database.
It allows database administrators to manage how and where data is stored on disk to improve performance, scalability, and maintenance.

The PRIMARY Filegroup

Every database has a default **PRIMARY** filegroup. Objects created without specifying a filegroup are stored in PRIMARY.

Filegroups in Partitioned Tables

In a partitioned table, filegroups can be used to store different partitions on separate storage locations enabling parallel I/O, targeted backups/restores, and simplified archival operations.

Enterprise Use

In large or enterprise systems, each partition is often mapped to its own filegroup located on a separate physical disk.
This configuration improves performance and supports better data management but requires administrative privileges.